

Theoretical Part

Assignment 2) B-spline Basis Functions

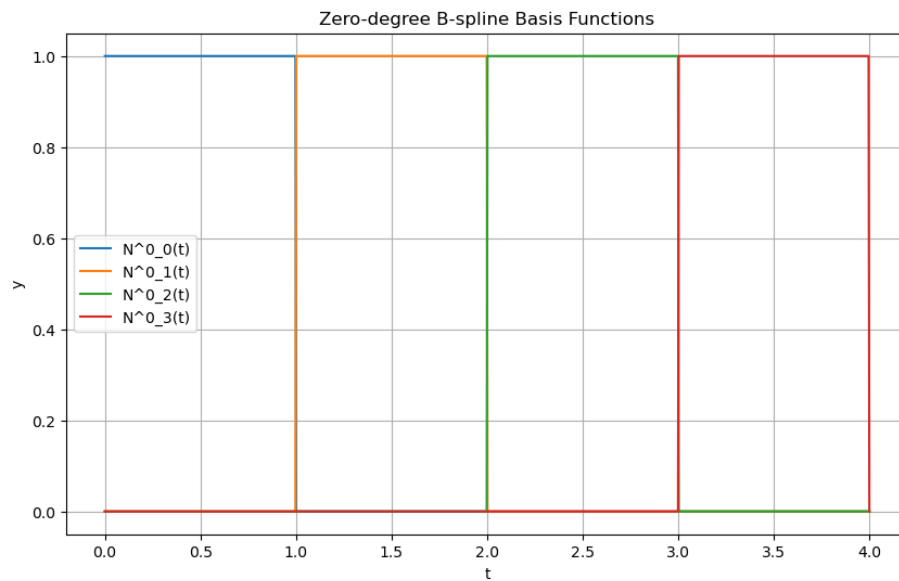
a)

```
import numpy as np
import matplotlib.pyplot as plt

U = [0, 1, 2, 3, 4]
t = np.linspace(0, 4, 1000)

NO_i = []
for u in range(len(U) - 1):
    y_i = (U[u] <= t) & (t < U[u + 1]).astype(int)
    NO_i.append(y_i)

plt.figure(figsize=(10, 6))
for i, N in enumerate(NO_i):
    plt.plot(t, N, label=f'N^0_{i}(t)')
plt.title('Zero-degree B-spline Basis Functions')
plt.xlabel('t')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```



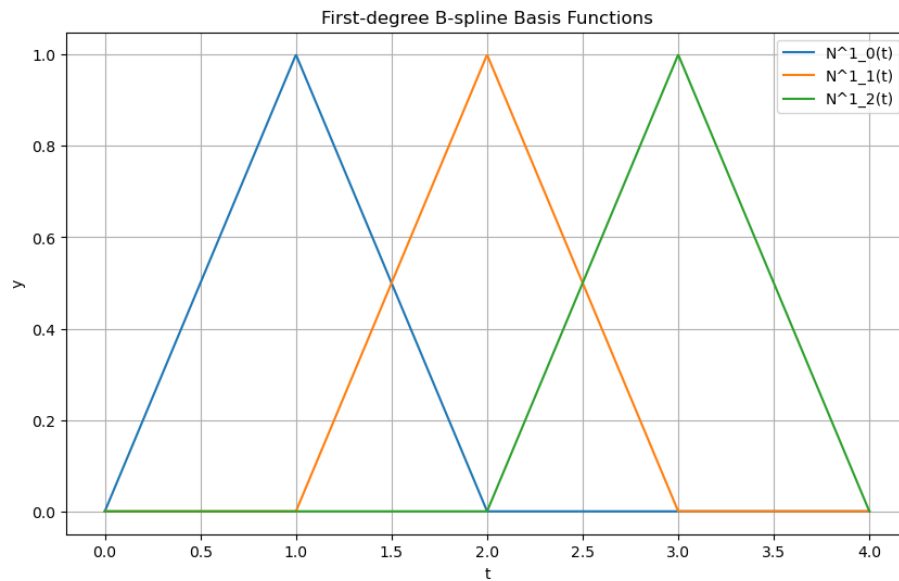
```
N1_i = []
for u in range(len(U) - 2):
```

```

y_i = (t - U[u]) / (U[u + 1] - U[u]) * NO_i[u] + (U[u + 1 + 1] - t) / (U[u + 1 + 1] - U[u]) * N1_i[u]
N1_i.append(y_i)

plt.figure(figsize=(10, 6))
for i, N in enumerate(N1_i):
    plt.plot(t, N, label=f'N^1_{i}(t)')
plt.title('First-degree B-spline Basis Functions')
plt.xlabel('t')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()

```

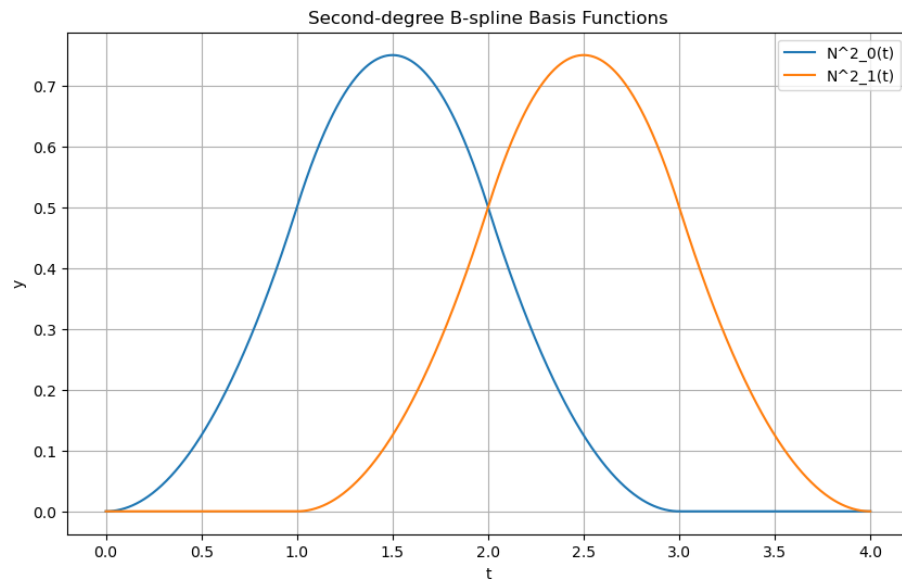


```

N2_i = []
for u in range(len(U) - 3):
    y_i = (t - U[u]) / (U[u + 2] - U[u]) * N1_i[u] + (U[u + 1 + 2] - t) / (U[u + 1 + 2] - U[u]) * N1_i[u + 1]
    N2_i.append(y_i)

plt.figure(figsize=(10, 6))
for i, N in enumerate(N2_i):
    plt.plot(t, N, label=f'N^2_{i}(t)')
plt.title('Second-degree B-spline Basis Functions')
plt.xlabel('t')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()

```

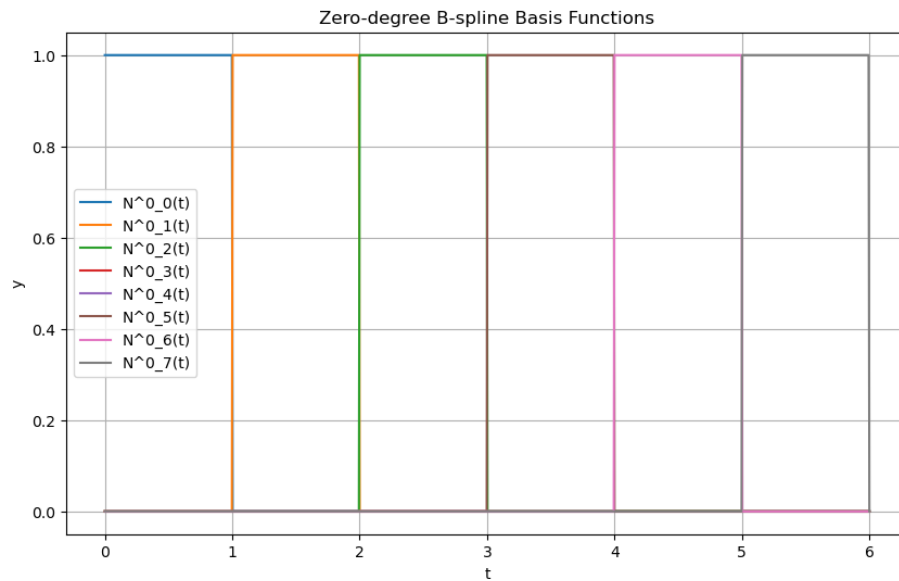


b)

```
U = [0, 1, 2, 3, 3, 3, 4, 5, 6]
t = np.linspace(0, 6, 1000)

NO_i = []
for u in range(len(U) - 1):
    y_i = (U[u] <= t) & (t < U[u + 1]).astype(int)
    NO_i.append(y_i)

plt.figure(figsize=(10, 6))
for i, N in enumerate(NO_i):
    plt.plot(t, N, label=f'N^0_{i}(t)')
plt.title('Zero-degree B-spline Basis Functions')
plt.xlabel('t')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```

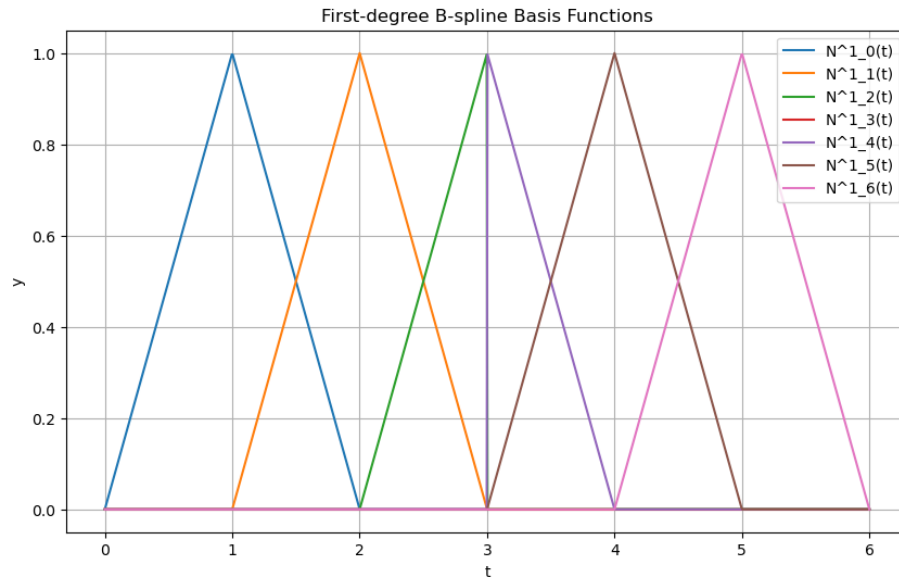


```

N1_i = []
for u in range(len(U) - 2):
    if (U[u + 1] - U[u]) == 0:
        left_term = np.zeros_like(t)
    else:
        left_term = (t - U[u]) / (U[u + 1] - U[u]) * NO_i[u]
    if (U[u + 1 + 1] - U[u + 1]) == 0:
        right_term = np.zeros_like(t)
    else:
        right_term = (U[u + 1 + 1] - t) / (U[u + 1 + 1] - U[u + 1]) * NO_i[u + 1]
    y_i = left_term + right_term
    N1_i.append(y_i)

plt.figure(figsize=(10, 6))
for i, N in enumerate(N1_i):
    plt.plot(t, N, label=f'N^1_{i}(t)')
plt.title('First-degree B-spline Basis Functions')
plt.xlabel('t')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()

```



```

N2_i = []
for u in range(len(U) - 3):
    if (U[u + 2] - U[u]) == 0:
        left_term = np.zeros_like(t)
    else:
        left_term = (t - U[u]) / (U[u + 2] - U[u]) * N1_i[u]
    if (U[u + 1 + 2] - U[u + 1]) == 0:
        right_term = np.zeros_like(t)
    else:
        right_term = (U[u + 1 + 2] - t) / (U[u + 1 + 2] - U[u + 1]) * N1_i[u + 1]
    y_i = left_term + right_term
    N2_i.append(y_i)

```

```

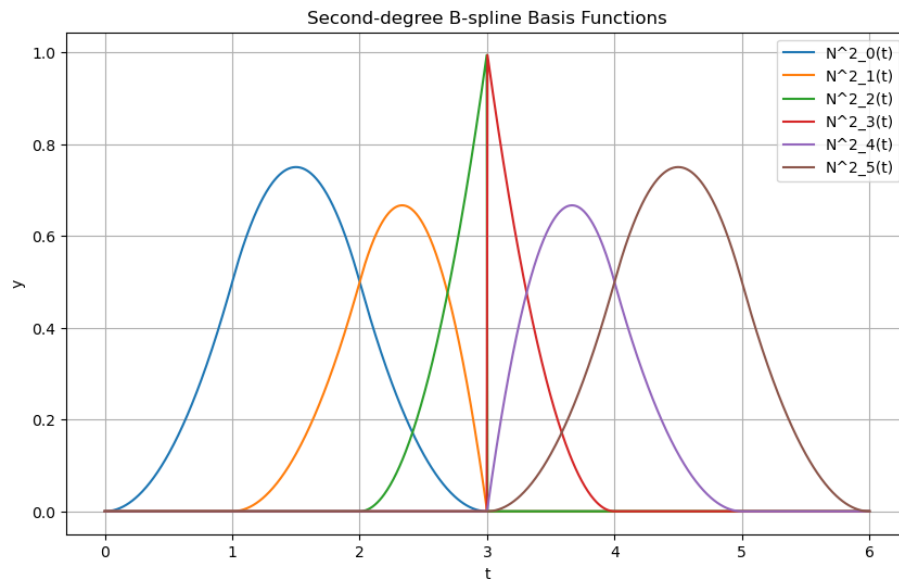
#y_i = (t - U[u]) / (U[u + 2] - U[u]) * N1_i[u] + (U[u + 1 + 2] - t) / (U[u + 1 + 2] - U[u + 1]) * N1_i[u + 1]

```

```

plt.figure(figsize=(10, 6))
for i, N in enumerate(N2_i):
    plt.plot(t, N, label=f'N^2_{i}(t)')
plt.title('Second-degree B-spline Basis Functions')
plt.xlabel('t')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt

# Knotenvektor
U = np.array([0,1,2,3,3,3,4,5,6])

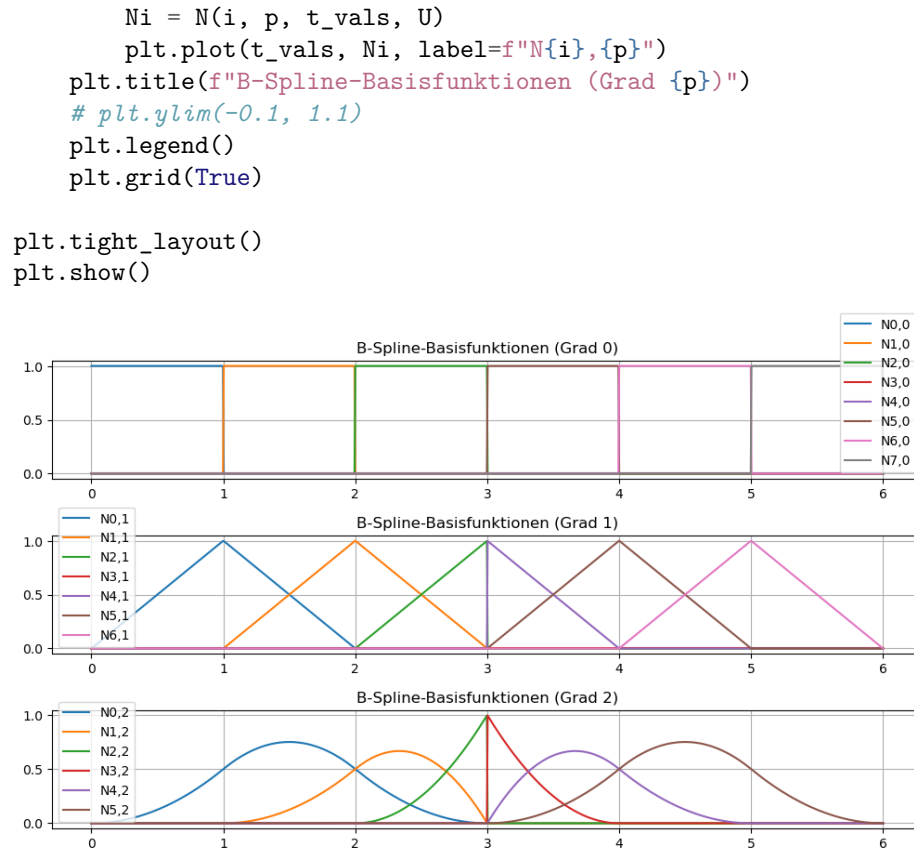
def N(i, p, t, U):
    """Rekursive Definition der B-Spline-Basisfunktion  $N_{\{i,p\}}(t)$ """
    if p == 0:
        return np.where((t >= U[i]) & (t < U[i+1]), 1.0, 0.0)
    else:
        left_denom = U[i+p] - U[i]
        right_denom = U[i+p+1] - U[i+1]
        left = np.zeros_like(t) if left_denom == 0 else (t - U[i]) / left_denom * N(i, p-1, t, U)
        right = np.zeros_like(t) if right_denom == 0 else (U[i+p+1] - t) / right_denom * N(i, p-1, t, U)
        return left + right

t_vals = np.linspace(0, 6, 1000)

plt.figure(figsize=(10, 6))

for p in range(3): # Grad 0, 1, 2
    n = len(U) - p - 1
    plt.subplot(3, 1, p+1)
    for i in range(n):

```



c)

$$N_{i,0}(u) = \begin{cases} 1, & u_i \leq u < u_{i+1} \\ 0, & \text{else} \end{cases}$$

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} \cdot N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} \cdot N_{i+1,p-1}(u)$$

$\boxed{N_{i,p}(u) \text{ is a degree } p \text{ polynomial:}}$

Induction:

Start of Induction: $p = 0$: $N_{i,0}(u)$ is a polynomial of form $c \cdot x^0$ with c being 1 if $u_i \leq u < u_{i+1}$ and 0 else

Induction Step: Given that $N_{i,p-1}(u)$ and $N_{i+1,p-1}(u)$ are polynomials of degree $p = p - 1$

Coefficients $a(u) = \frac{u - u_i}{u_{i+p} - u_i}$ and $b(u) = \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}}$ are polynomials of degree $p = 1$

If we now multiply $a(u)$ ($p = 1$) with $N_{i,p-1}(u)$ ($p = p - 1$) and add $b(u)$ ($p = 1$) multiplied with $N_{i+1,p-1}(u)$ ($p = p - 1$), we get a polynomial of degree $1 + (p - 1) = p$ $\square$

For all i, p and u , $N_{i,p}(u)$ is non negative:

Induction: Start of Induction: $p = 0 : N_{i,0}(u)$ is either 1 or 0 $\implies$ non-negative

Induction Step: Given that $N_{i,p-1}(u)$ and $N_{i+1,p-1}(u)$ are non-negative:

Coefficient $a(u) = \frac{u-u_i}{u_{i+p}-u_i}$ is positive because nominator $u - u_i$ and denominator $u_{i+p} - u_i$ are positive, due to definition that $u > u_i$ and $u_{i+p} > u_i$

Furthermore coefficient $b(u) = \frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}}$ is positive because $u_{i+p+1} - u$ is positive due to definition of $u_{i+p+1} > u$ and $u_{i+p+1} - u_{i+1}$ is positive due to $u_{i+p+1} > u_{i+1}$

So we have $\underbrace{a(u)}_{\text{positive}} \cdot \underbrace{N_{i,p-1}(u)}_{\text{positive}} + \underbrace{b(u)}_{\text{positive}} \cdot \underbrace{N_{i+1,p-1}(u)}_{\text{positive}}$ and in total that is positive again. $\square$

d)

$U_1 = \{0, 0, 1, 1\} :$

$N_{0,0}(u) = 0$, $N_{1,0}(u) = 1$ for $u \in [0, 1)$ and $N_{2,0} = 0$

$$N_{0,1}(u) = \underbrace{\frac{u-u_0}{u_1-u_0} \cdot N_{0,0} + \frac{u_2-u}{u_2-u_1} \cdot N_{1,0}}_{\substack{/0 \implies 0 \\ 0}} = \frac{1-u}{1-0} \cdot N_{1,0} = 1 - u$$

$$N_{1,1}(u) = \frac{u-u_1}{u_2-u_1} \cdot N_{1,0} + \underbrace{\frac{u_3-u}{u_3-u_2} \cdot N_{2,0}}_{\substack{/0 \implies 0 \\ 0}} = \frac{u-0}{1-0} \cdot N_{1,0} = u$$

$$B_{0,1}(u) = 1 \cdot u^0 \cdot (1-u)^1 = 1 - u = N_{0,1}(u)$$

$$B_{1,1}(u) = 1 \cdot u^1 \cdot (1-u)^0 = u = N_{1,1}(u)$$

$U_1 = \{0, 0, 0, 1, 1, 1\} :$

$\boxed{p=0} :$ $N_{0,0}(u) = 0$, $N_{1,0}(u) = 0$, $N_{2,0}(u) = 1$ for $u \in [0, 1)$, $N_{3,0} = 0$ and $N_{4,0} = 0$

$p = 1 :$

$$N_{0,1}(u) = \underbrace{\frac{u-u_0}{u_1-u_0} \cdot N_{0,0} + \frac{u_2-u}{u_2-u_1} \cdot N_{1,0}}_{\substack{/0 \Rightarrow 0 \\ 0}} = \frac{1-u}{1-0} \cdot N_{1,0} = 0$$

$$N_{1,1}(u) = \underbrace{\frac{u-u_1}{u_2-u_1} \cdot N_{1,0}}_0 + \frac{u_3-u}{u_3-u_2} \cdot N_{2,0} = \frac{1-u}{1-0} \cdot N_{2,0} = 1-u$$

$$N_{2,1}(u) = \frac{u-u_2}{u_3-u_2} \cdot N_{2,0} + \underbrace{\frac{u_4-u}{u_4-u_3} \cdot N_{3,0}}_0 = \frac{u-0}{1-0} \cdot N_{2,0} = u$$

$$N_{3,1}(u) = \underbrace{\frac{u-u_3}{u_4-u_3} \cdot N_{3,0}}_0 + \underbrace{\frac{u_4-u}{u_4-u_3} \cdot N_{4,0}}_0 = 0$$

$$\boxed{p=2}:$$

$$N_{0,2}(u) = \underbrace{\frac{u-u_0}{u_2-u_0} \cdot N_{0,1}}_0 + \frac{u_3-u}{u_3-u_1} \cdot N_{1,1} = (1-u) \cdot (1-u) = (1-u)^2$$

$$N_{1,2}(u) = \frac{u-u_1}{u_3-u_1} \cdot N_{1,1} + \frac{u_4-u}{u_4-u_2} \cdot N_{2,1} = u \cdot (1-u) + (1-u) \cdot u = 2u(1-u)$$

$$N_{2,2}(u) = \frac{u-u_2}{u_4-u_2} \cdot N_{2,1} + \underbrace{\frac{u_5-u}{u_5-u_3} \cdot N_{3,1}}_0 = u \cdot u = u^2$$

$$B_{0,2}(u) = \binom{2}{0} \cdot u^0 \cdot (1-u)^2 = (1-u)^2 = N_{0,2}(u)$$

$$B_{1,2}(u) = \binom{2}{1} \cdot u^1 \cdot (1-u)^1 = 2u(1-u) = N_{1,2}(u)$$

$$B_{2,2}(u) = \binom{2}{2} \cdot u^2 \cdot (1-u)^0 = u^2 = N_{2,2}(u)$$